COURSE PROJECT SPECIFICATION

Course: Distributed Systems

1. Project Objectives

Students will apply knowledge of distributed system architecture, inter-process communication, naming and service discovery, synchronization, concurrency control, data consistency, replication, fault tolerance, recovery, and security to build a distributed system prototype.

Upon completion of the project, students should demonstrate the ability to:

- Analyze a problem and identify distributed-system issues that need to be addressed.
- Select appropriate architectures, protocols, and algorithms.
- Build independent processes, peers, or services that communicate over a network.
- Handle multiple concurrent requests or users.
- Manage state and maintain consistency.
- Detect, handle, and recover from selected failures.
- Test and evaluate the system.
- Write a technical report, demonstrate, and defend the solution.

2. Project Format

- Students work in groups of no more than 3 members.
- Each group selects one of the four specified topics.
- Groups selecting the same topic may use different technologies and solutions.
- Each member must be responsible for a specific part of the work and provide evidence of contribution on GitHub.
- Each member must understand the overall architecture and be able to explain their assigned work.
- Groups must register their topic, members, preliminary architecture, and intended technologies with the instructor.

3. List of Topics

Code	Topic	Main Distributed Systems Concepts
DS01	P2P chat system simulating Skype	P2P, super-peer/relay, NAT, peer discovery, reliable communication
DS02	P2P file-sharing system simulating BitTorrent	P2P, tracker, data chunking and replication, parallel downloading, churn

Code	Topic	Main Distributed Systems Concepts
DS03	Ride-hailing microservices system simulating Grab/Uber	REST, message queue, distributed transactions, eventual consistency, fault tolerance
DS04	Collaborative document editing system simulating Google Docs	Real-time synchronization, causality, OT/CRDT, conflict handling

4. General Requirements for All Topics

4.1. Distributed Nature

The system must consist of at least three independent processes, peers, clients, or services.

Components must:

- Run as separate processes, applications, virtual machines, or containers.
- Communicate over a network using sockets, HTTP, WebSocket, gRPC, message queues, or equivalent protocols.
- Have unique identities.
- Be capable of handling multiple concurrent connections or requests.
- Not use global variables, shared memory, or shared files as substitutes for network communication.
- Produce logs showing nodes, timestamps, event types, and processing results.

The system may be deployed:

- On multiple computers in a LAN.
- On computers located in different Internet networks.
- On virtual machines or cloud servers.
- In containers with independent processes and networks.
- Using a combination of the above approaches.

4.2. Architecture and Communication

Groups must design and present:

- Components and the responsibilities of each component.
- Naming and identification mechanisms.
- Peer discovery or service discovery mechanisms.
- Communication protocols.
- Message formats.
- How distributed state is managed.
- How online/offline components are detected.
- Timeouts, acknowledgements, and retries.

- Mechanisms for handling duplicate or out-of-order messages.
- Constraints related to performance, scalability, and availability.

4.3. Concurrency and Consistency

The system must include at least one scenario involving concurrent actions by multiple processes or users.

Groups must:

- Identify possible conflicts.
- Define the intended consistency model.
- Design a concurrency-control mechanism.
- Test the system using concurrent operations.
- Demonstrate that the final state of the system satisfies the requirements.

Depending on the topic, groups may use:

- Sequence numbers.
- Lamport logical clocks.
- Vector clocks.
- Version numbers.
- Idempotency keys.
- Optimistic concurrency control.
- Event logs.
- Saga.
- Operational Transformation.
- CRDT.

4.4. Fault Tolerance and Recovery

Groups must identify a failure model and demonstrate at least two failure scenarios, such as:

- A peer or service unexpectedly leaves the network.
- A client loses connectivity.
- A super-peer or relay does not respond.
- A message is lost, duplicated, or arrives out of order.
- A file download is interrupted.
- A service fails to process a message.
- A document-editing client loses connectivity.

The system must use appropriate mechanisms such as:

- Timeout.
- Bounded retry.

- Acknowledgement.
- Heartbeat or lease.
- Deduplication.
- Checkpointing or intermediate state persistence.
- Reconnection.
- Resume.
- Failover.
- Dead-letter queue.
- Circuit breaker.
- Compensating transactions.

4.5. Basic Security

- Authenticate users, peers, or services.
- Enforce access control for resources.
- Validate and sanitize input data.
- Do not store passwords in plaintext.
- Do not commit tokens, passwords, or secret keys to GitHub.
- Use environment variables for secret configuration.
- Limit message or file sizes.
- Log important activities.
- The use of TLS or message encryption is encouraged.

5. Topic DS01: Build a P2P Chat System Simulating Skype

5.1. Objective

Build a system that enables multiple peers to exchange messages directly over a network. Each peer must be able both to initiate connections and to accept connections from other peers.

The system must operate in a WAN/Internet environment, including situations in which peers are behind NAT, use private IP addresses, or cannot open ports for direct incoming connections.

To address this issue, the group must design and deploy one of the following solutions:

- A super-peer with a public IP address.
- A relay server.
- A system consisting of multiple super-peers/relays.
- An equivalent solution that can support connectivity between peers over the Internet.

A bootstrap server or signaling server may be used for registration, peer discovery, and exchange of connection-establishment information.

5.2. Minimum Architecture

The system must include:

- A bootstrap/signaling server.
- At least one super-peer or relay server that can be accessed from the Internet.
- At least three peers.
- A component for storing local chat history.
- A command-line, desktop, or web interface for each peer.

The architecture must support two communication modes:

Mode 1: Direct P2P Connection

If two peers can establish a direct connection, messages are transmitted directly between the peers without passing through the relay server.

Mode 2: Connection through a Super-Peer/Relay

If two peers cannot connect directly due to NAT, firewalls, or network limitations, messages are transmitted through a super-peer/relay server.

Groups must clearly define:

- Conditions for selecting a direct connection.
- Conditions for switching to a relay.
- How peers detect that direct connection establishment has failed.
- How the relay identifies the destination peer.
- How sessions and peer-to-connection mappings are maintained.
- How relay disconnection is handled.

A system in which the bootstrap server secretly acts as a centralized chat server but is described as P2P will not be accepted. If all messages always pass through a relay, the group must explain that it is a relay-based architecture and clearly analyze its limitations compared with direct P2P communication.

5.3. Joining the Network and Peer Discovery

- Each peer must have a unique Peer ID.
- A peer registers with the bootstrap/signaling server when joining the network.
- Registration information includes Peer ID, display name, status, and connectivity information.
- The bootstrap server provides a list of currently online peers.
- Peers may send heartbeats to maintain their status.
- The system updates when a peer joins, leaves, or loses connectivity.

- Expired peer information must be removed after a timeout period.

5.4. Connection Establishment in a WAN Environment

The system must perform the following process:

1. The source peer looks up the destination peer.
2. The source peer attempts to establish a direct connection.
3. If the connection succeeds, the two peers switch to direct P2P mode.
4. If the connection fails or times out, the source peer requests support from a super-peer/relay.
5. The relay establishes or uses existing connections with both peers.
6. Messages are forwarded to the destination peer.
7. Both peers are aware of whether the communication path is direct or relay-based.

Groups may use:

- TCP sockets with a relay server.
- WebSocket with peers maintaining outbound connections.
- WebRTC DataChannel combined with signaling and TURN.
- Reverse connections.
- Tunneling.
- An equivalent protocol designed by the group.

If WebRTC or an existing library is used, the group must still explain signaling, NAT traversal, STUN/TURN, and the fallback mechanism through a relay.

5.5. Direct Chat

- Users select the peer they want to contact.
- The system prioritizes direct messaging when possible.
- If a direct connection cannot be established, the system automatically uses a relay.
- A peer can send and receive messages simultaneously.
- The interface must display message-delivery status.
- Displaying the transmission method, `DIRECT` or `RELAY`, is encouraged.
- Chat history is stored locally.

5.6. Group Chat

- Users can create groups.
- Add or remove members.
- Send messages to all members of a group.
- Support members located in different networks.
- Messages may be transmitted directly, through a super-peer, or using a combination of both.

- Peers must maintain a consistent group membership list.
- The system must define how offline members are handled.

5.7. Message Structure

Each message must contain at least:

- Message ID.
- Sender ID.
- Receiver ID or Group ID.
- Session ID.
- Message type.
- Timestamp or logical timestamp.
- Content.
- Delivery mode: direct or relay.
- Status or acknowledgement information.

Message types may include:

- `REGISTER`
- `HEARTBEAT`
- `PEER_LIST`
- `CONNECT_REQUEST`
- `CONNECT_RESPONSE`
- `CHAT_MESSAGE`
- `GROUP_MESSAGE`
- `ACK`
- `DELIVERY_ERROR`
- `DISCONNECT`

Groups may choose JSON, Protocol Buffers, or another suitable format.

5.8. Reliable Message Delivery

The system must include:

- Unique Message IDs.
- Acknowledgements.
- Timeouts.
- Bounded retries.
- Detection and removal of duplicate messages.
- Reasonable ordering of messages within each conversation.
- Notifications when a message cannot be delivered.
- Differentiation between sent, forwarded, and received statuses, if implemented.

Groups must clearly describe the delivery semantics, for example:

- At-most-once.
- At-least-once combined with deduplication.
- Best effort with application-layer acknowledgement.

5.9. Requirements for the Super-Peer/Relay

The super-peer or relay server must:

- Have an address or domain name accessible from the Internet.
- Accept connections from multiple peers.
- Maintain mappings between Peer IDs and active connections.
- Forward messages to the correct peer.
- Support multiple concurrent connections.
- Detect disconnected peers.
- Not alter message content without authorization.
- Produce logs for registration, connection, and forwarding activities.
- Limit message size and validate message formats.
- Prevent one peer from impersonating another peer's Peer ID.

Groups must evaluate:

- Whether the relay becomes a single point of failure.
- How the relay affects latency.
- How relay bandwidth affects scalability.
- What happens when the relay stops operating.
- How multiple relays could be added.

5.10. Mandatory Demonstration Scenarios

1. Deploy the bootstrap/signaling server and relay on a server accessible from the Internet.
2. Start at least three peers.
3. Peers register and discover one another.
4. Two peers that can connect directly perform direct P2P chat.
5. Display logs proving that messages do not pass through the relay.
6. Two peers from different networks or behind NAT chat through the relay.
7. Display logs on peers and the relay to prove the communication path.
8. Three peers join a group and exchange messages.
9. Two peers send messages nearly simultaneously.
10. Abruptly stop one peer and demonstrate that the system updates its status.
11. Break a direct connection and demonstrate that the system switches to relay mode.
12. Restart a peer and restore the appropriate state.

If real conditions do not permit the use of multiple Internet connections, groups may simulate separate network domains using virtual machines or Docker networks. However, the relay must

still be deployed as an independent component, and there must be evidence that peers cannot establish direct connections but can still chat through the relay.

5.11. Advanced Features

- Multiple super-peers or relay servers.
- Relay selection based on latency or load.
- Automatic switching to another relay when the current relay fails.
- Election of a publicly reachable peer as a super-peer.
- State replication among relays.
- Store-and-forward for offline peers.
- End-to-end encryption so the relay cannot read message content.
- DHT instead of centralized bootstrap.
- File transfer between peers.
- NAT traversal using STUN.
- WebRTC and TURN.
- Voice or video calls at prototype level.
- Load testing with a large number of simulated peers.

6. Topic DS02: Build a P2P File-Sharing System Simulating BitTorrent

6.1. Objective

Build a system that allows files to be split into chunks and downloaded in parallel from multiple peers. A peer that has downloaded a chunk must be able to provide that chunk to other peers.

The tracker manages only metadata and peer lists; it must not provide the entire file content.

6.2. Minimum Components

- A tracker or bootstrap server.
- At least three peers.
- A metadata generator.
- Components for splitting and merging chunks.
- A mechanism for concurrent chunk downloading and uploading.

6.3. Functional Requirements

Metadata

Metadata must contain:

- File ID.
- File name and size.
- Chunk size and number of chunks.
- Hash of each chunk.
- Hash of the entire file.
- Tracker information.

Sharing and Peer Discovery

- The seeder splits the file into chunks.
- The seeder registers the file with the tracker.
- The tracker manages peers that possess all or part of the file.
- Peers announce the list of chunks they currently have.

File Downloading

- Download from at least two peers.
- Download multiple chunks in parallel.
- A peer can upload while downloading.
- A failed chunk is requested again from another peer.
- Limit the number of connections or download threads.

Integrity and Recovery

- Verify the hash of each chunk.
- Reject invalid chunks.
- Merge chunks in the correct order.
- Verify the final file hash.
- Continue downloading after a source peer leaves the network.
- Resume download after a program restart is encouraged.

6.4. Mandatory Demonstration Scenarios

- Create metadata and share a file.
- Download a file in parallel from at least two peers.
- Demonstrate that a peer can upload before completing its own download.
- Stop one source during downloading.
- Switch to another source.
- Detect an invalid chunk.
- Merge the file and verify the final hash.

6.5. Advanced Features

- Rarest-first.
- Choking/unchoking or tit-for-tat.

- DHT.
- Magnet links.
- Replicated tracker.
- Bandwidth limiting.
- Resume download.
- Encrypted connections.
- Churn simulation and chunk availability evaluation.

7. Topic DS03: Build a Ride-Hailing Microservices System Simulating Grab/Uber

7.1. Objective

Build a ride-hailing system using a microservices architecture. Services must be deployed independently and communicate using RESTful APIs and message queues.

7.2. Minimum Services

The system must include at least four business microservices selected from:

- Identity/User Service.
- Driver/Vehicle/Location Service.
- Trip/Booking Service.
- Matching Service.
- Pricing Service.
- Simulated Payment Service.
- Notification Service.

An API Gateway does not count as a business microservice.

Each service must have its own data or separate data schema and must not directly access another service's database.

7.3. Functional Requirements

- User registration, login, and passenger/driver role assignment.
- Drivers update their status and simulated location.
- Passengers create ride requests.
- Find and select suitable drivers.
- Drivers accept or reject trips.
- A driver must not be assigned to two trips simultaneously.
- Manage the trip lifecycle.
- Calculate fare and simulate payment.

- Send notifications to passengers and drivers.

The following are mandatory:

- At least one REST or gRPC communication flow.
- At least one asynchronous communication flow using a message queue.
- Event IDs or idempotency keys.
- Retries.
- Deduplication.
- A dead-letter queue or error-message storage.
- Correlation IDs.
- One distributed transaction using Saga or compensating actions.

7.4. Mandatory Demonstration Scenarios

- Start services independently.
- Drivers become available.
- A passenger requests a ride.
- The Matching Service selects a driver.
- The driver accepts and completes the trip.
- Two passengers simultaneously request the same driver.
- Stop one service during a business flow.
- Demonstrate retry, queue handling, or compensation.
- Restart the service and continue processing messages.

7.5. Advanced Features

- API Gateway and service discovery.
- Circuit breaker.
- Outbox pattern.
- Distributed tracing.
- Centralized logging.
- Multiple-instance scaling.
- Dynamic pricing.
- Geospatial matching.
- Docker Compose, Kubernetes, or cloud deployment.
- Service-monitoring dashboard.

8. Topic DS04: Build a Collaborative Document Editing System Simulating Google Docs

8.1. Objective

Build a system that allows multiple users to edit a document in near real time. Clients must converge to the same document content after concurrent operations.

A solution that simply overwrites the entire document based on “last writer wins” will not be accepted.

8.2. Minimum Components

- A collaboration server.
- At least three clients.
- WebSocket or an equivalent real-time channel.
- A document storage component.
- An operation and version-management mechanism.

8.3. Functional Requirements

- Create, open, edit, and save documents.
- Display active users.
- Synchronize insert and delete operations.
- Handle concurrent editing by multiple clients.
- Each operation must include an ID, Client ID, Document ID, operation type, and version information.
- Use OT, CRDT, or an equivalent mechanism.
- Handle duplicate or out-of-order operations.
- Synchronize new clients joining a document.
- Re-synchronize clients after connection loss.
- Restore documents after a server restart.

8.4. Mandatory Demonstration Scenarios

- Three clients open the same document.
- One client edits, and the other clients receive updates.
- Two clients insert content at the same position simultaneously.
- One client inserts while another deletes.
- Demonstrate convergence among clients.

- Disconnect one client.
- The remaining clients continue editing.
- The disconnected client reconnects and synchronizes.
- Restart the server and restore the document.

8.5. Advanced Features

- Offline editing.
- Decentralized CRDT.
- Display of user cursors and selections.
- Collaborative undo/redo.
- Version history.
- Comments and suggestions.
- Collaboration-server replication.
- Snapshots combined with operation logs.
- Owner/editor/viewer access control.

9. Testing and Evaluation

Each group must perform at least:

- 8 functional tests.
- 2 concurrency or conflict tests.
- 2 fault and recovery tests.
- 2 security or invalid-data tests.
- 1 performance or scalability experiment.

Possible evaluation metrics include:

- Message or request latency.
- Throughput.
- Delivery success rate.
- Failure-detection time.
- Relay-switching time.
- Recovery time.
- File download time.
- Document convergence speed.
- Number of concurrent connections.
- CPU, memory, and bandwidth usage.

10. Required Deliverables

10.1. GitHub Repository

The repository must include at least:

```
README.md
src/
deploy/
tests/
scripts/
docs/
.env.example
```

Where:

- `README.md`: project description, architecture, environment requirements, installation instructions, execution instructions, and demonstration scenarios.
- `src/`: source code.
- `deploy/`: Docker, server, or deployment configuration.
- `tests/`: test code and test scenarios.
- `scripts/`: scripts for startup, workload generation, or fault injection.
- `docs/report.pdf`: project report.
- `docs/slides.pdf`: presentation slides, if available.
- `.env.example`: sample configuration without secret information.

The repository must:

- Include a commit history from each member.
- Include redeployment instructions.
- Include a task-assignment table.
- Include a final tag or release, for example `v1.0.0`.
- Grant access to the instructor if the repository is private.

10.2. Project Report

Recommended length: 20–30 pages, excluding appendices.

The report includes:

1. Cover page.
2. Group information and task assignment.
3. Abstract.
4. Problem description and requirements.
5. Distributed-system issues.

6. Analysis and solution selection.
7. System architecture.
8. Component and protocol design.
9. Identification and discovery.
10. Concurrency, synchronization, and consistency.
11. Fault tolerance and recovery.
12. Security.
13. Implementation.
14. Testing and results.
15. Performance evaluation.
16. Limitations and future work.
17. Conclusion.
18. References.
19. Appendices.

11. Presentation and Demonstration

Suggested duration:

- Problem and architecture introduction: 3–5 minutes.
- Demonstration: 7–10 minutes.
- Oral examination: approximately 5 minutes.

The demonstration must:

- Use at least three nodes, peers, clients, or services.
- Show network communication.
- Include one concurrent scenario.
- Include at least one live failure scenario.
- Display logs or traces to explain system behavior.
- Involve all group members.

12. Cases That Do Not Meet the Requirements

- The system runs in only one process.
- Components do not communicate over a network.
- The P2P chat system works only on localhost or in a LAN and has no super-peer, relay, or equivalent WAN solution.
- The bootstrap server forwards all messages but is not analyzed as a relay.
- The BitTorrent tracker provides the entire file content.

- Microservices share a database and directly access each other's data.
- The collaborative editing system uses last-write-wins for the entire document.
- No concurrent scenario is demonstrated.
- No failure and recovery scenario is demonstrated.
- The source code cannot run according to the instructions.
- The report is not submitted or the product is not demonstrated.
- A member cannot prove their individual contribution.

13. Academic Integrity

- Open-source libraries, frameworks, and source code may be used with proper citation.
- A complete existing product may not be used with only interface changes.
- Core mechanisms must be deliberately implemented, configured, and clearly explained by the group.
- Sources must be cited for algorithms, source code, figures, and references.
- If AI tools are used, the group must verify, understand, and take responsibility for the resulting product.
- Plagiarism or academic dishonesty will be handled according to Academy regulations.

Important Notes:

- Mid-term Exam: November 16, 2026 (Tentative, subject to change)
- Final report and source code submission deadline: November 22, 2026. Submit via Github repository.
- Project demonstration and oral examination: November 23, 2026, and November 30, 2026
- Group registration at (make sure you fill in the right student ID. Incorrect student ID can lead to missing score): https://docs.google.com/spreadsheets/d/16kScis-mmU_3uDepWRRfm-Ox3AhMBIqnIT2qFxBDdxY/edit?usp=sharing